Reduced-Order Modelling

Scientific Machine Learning and Simulation Intelligence

Reduced-Order Modelling

FEM for advection diffusion

Last time we worked on a simple case involving both advection and diffusion:

$$\mathbf{M}\frac{\mathbf{u}^{t+1}-\mathbf{u}^{t}}{\Delta t}+\mathbf{C}\mathbf{u}^{t}=-\frac{1}{\mathrm{Pe}}\mathbf{K}\mathbf{u}^{t}$$

We solve:

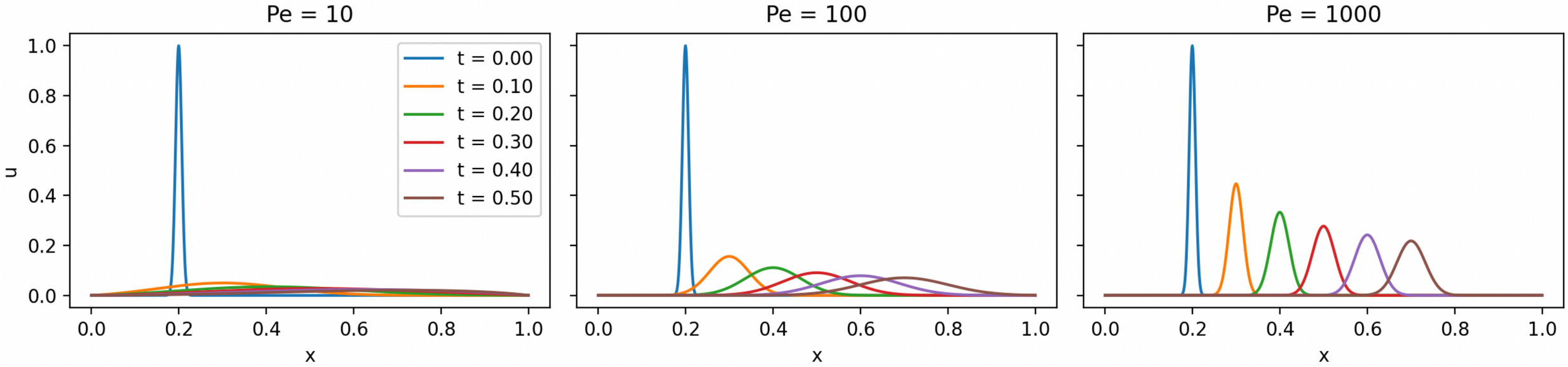
$$\boxed{\mathbf{Z}^{T}\mathbf{M}\mathbf{Z}}\frac{\mathbf{u}_{N}^{t+1}-\mathbf{u}_{N}^{t}}{\Delta t}+\boxed{\mathbf{Z}^{T}\mathbf{C}\mathbf{Z}}\mathbf{u}_{N}^{t}=-\frac{1}{\mathrm{Pe}}\boxed{\mathbf{Z}^{T}\mathbf{K}\mathbf{Z}}\mathbf{u}_{N}^{t}.$$

Reduced-Order Modelling

FEM for advection diffusion

We noticed a few interesting things regarding low-dimensional representations of these systems:

- ROMs perform significantly better for diffusion dominated problems: in general, cases where the intersection of the supports of the snapshots is often non-zero

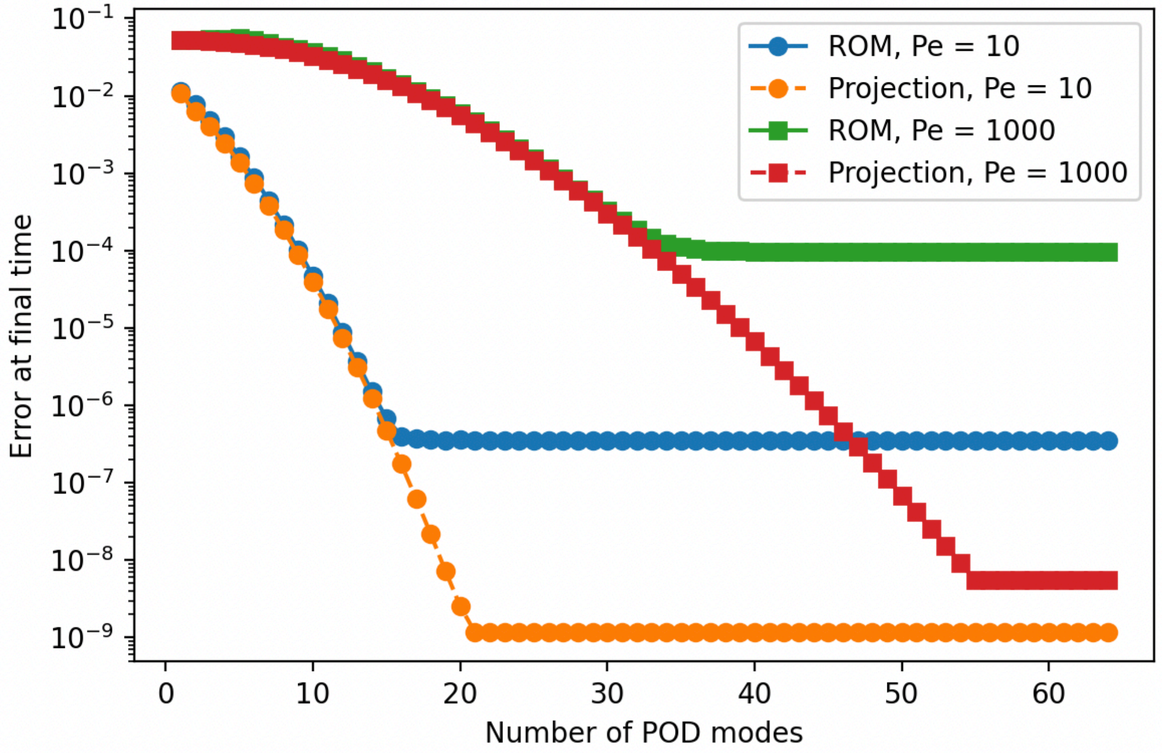


Reduced-Order Modelling

FEM for advection diffusion

We noticed a few interesting things regarding low-dimensional representations of these systems:

- Increasing the number of retained modes produced better and better results, leading to a fast convergence towards the expected solution of the problem.



We also noticed that resolving the reduced equations introduce further numerical errors and as we increase the number of modes the difference between projection and ROM prediction saturates due to those errors

Reduced-Order Modelling

FEM for advection diffusion

We noticed a few interesting things regarding low-dimensional representations of these systems:

- We saw, from an implementation point of view how to implement intrusive ROMs with particular care on the offline and online stages:

```
def pod(S):
    lam,W = la.eigh(S.T @ M @ S)
    lam,W = lam[::-1],W[:,::-1]
    p = lam > 1e-14*lam[0]
    lam,W = lam[p],W[:,p]
    e = np.cumsum(lam/lam.sum())
    N = np.searchsorted(e,energy_target)+1
    return S@W[:,:N]/np.sqrt(lam[:N]),N,lam,e
```

```
Z10,N10,lam10,e10 = pod(S10)
Z1000,N1000,lam1000,e1000 = pod(S1000)
```

```
def rom(Z,S,Pe):
    A = Z.T @ (K/Pe+C) @ Z
    a = Z.T @ M @ S[:,0]
    start = time.perf_counter()
    for _ in range(n_steps):
        a -= dt*(A@a)
    return Z@a,time.perf_counter()-start
```

Reduced-Order Modelling

FEM for advection diffusion

We noticed a few interesting things regarding low-dimensional representations of these systems:

- We saw, from an implementation point of view how to implement intrusive ROMs with particular care on the offline and online stages:

```
def pod(S):
    lam,W = la.eigh(S.T @ M @ S)
    lam,W = lam[::-1],W[:,::-1]
    p = lam > 1e-14*lam[0]
    lam,W = lam[p],W[:,p]
    e = np.cumsum(lam/lam.sum())
    N = np.searchsorted(e,energy_target)+1
    return S@W[:,:N]/np.sqrt(lam[:N]),N,lam,e
```

```
Z10,N10,lam10,e10 = pod(S10)
Z1000,N1000,lam1000,e1000 = pod(S1000)
```

```
def rom(Z,S,Pe):
    A = Z.T @ (K/Pe+C) @ Z
    a = Z.T @ M @ S[:,0]
    start = time.perf_counter()
    for _ in range(n_steps):
        a -= dt*(A@a)
    return Z@a,time.perf_counter()-start
```

Reduced-Order Modelling

FEM for advection diffusion

Now, we do have our low dimensional representation which is identified by the coefficients of the new basis we picked. We need an equation to evolve those coefficients. In projection based ROMs this come from the projection of the equations themselves.
Instead of solving the FE setting we introduced:

$$\mathbf{M}\frac{\mathbf{u}^{t+1}-\mathbf{u}^{t}}{\Delta t}+\mathbf{C}(\mathbf{u}^{t})\mathbf{u}^{t}=-\frac{1}{\mathrm{Pe}}\mathbf{K}\mathbf{u}^{t}$$

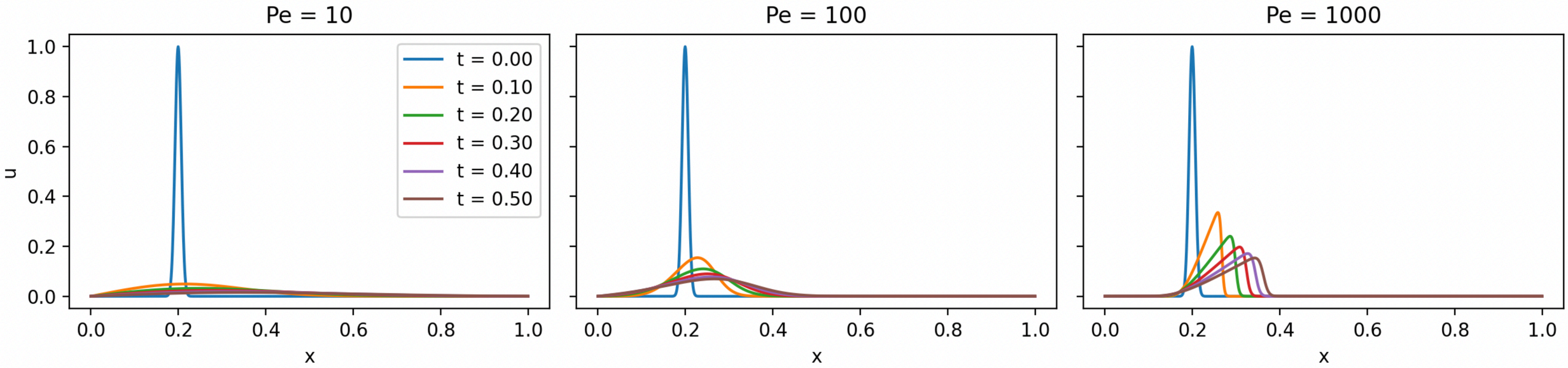
We solve:

$$\mathbf{Z}^{T}\mathbf{M}\mathbf{Z}\frac{\mathbf{u}_{N}^{t+1}-\mathbf{u}_{N}^{t}}{\Delta t}+\mathbf{Z}^{T}\mathbf{C}(\mathbf{Z}\mathbf{u}_{N}^{t})\mathbf{Z}\mathbf{u}_{N}^{t}=-\frac{1}{\mathrm{Pe}}\mathbf{Z}^{T}\mathbf{K}\mathbf{Z}\mathbf{u}_{N}^{t}.$$

Reduced-Order Modelling

FEM for advection diffusion

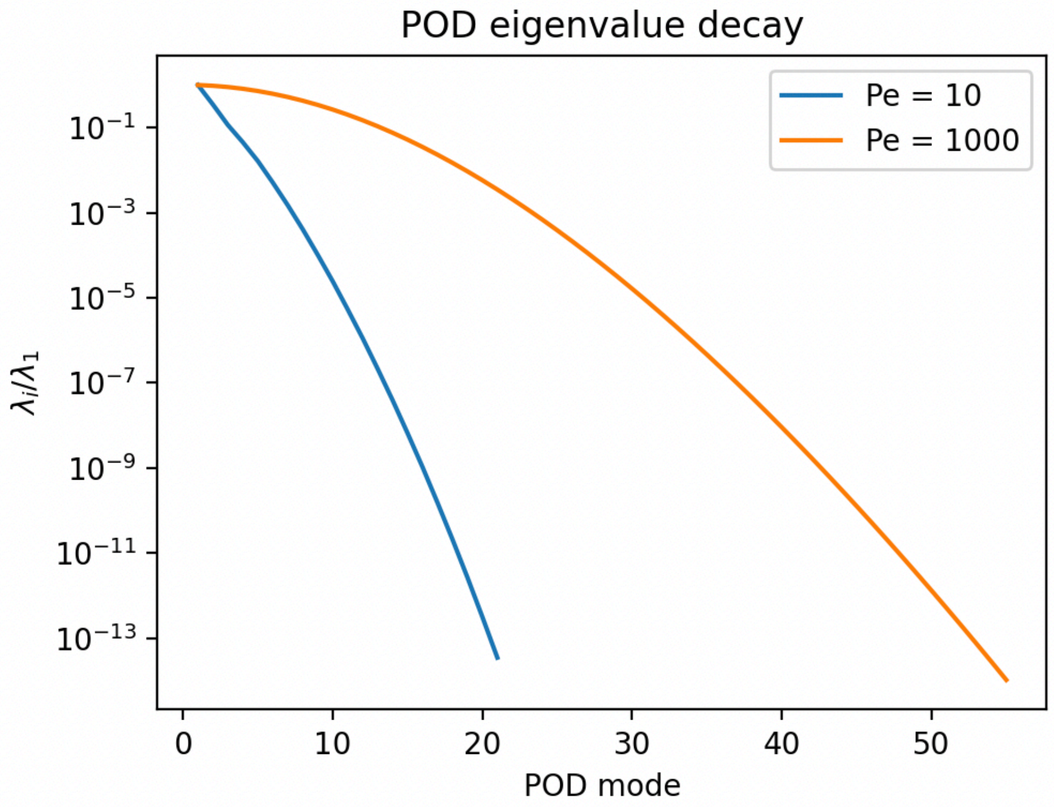
We noticed as well some interesting type of dynamics when we add non-linearity:

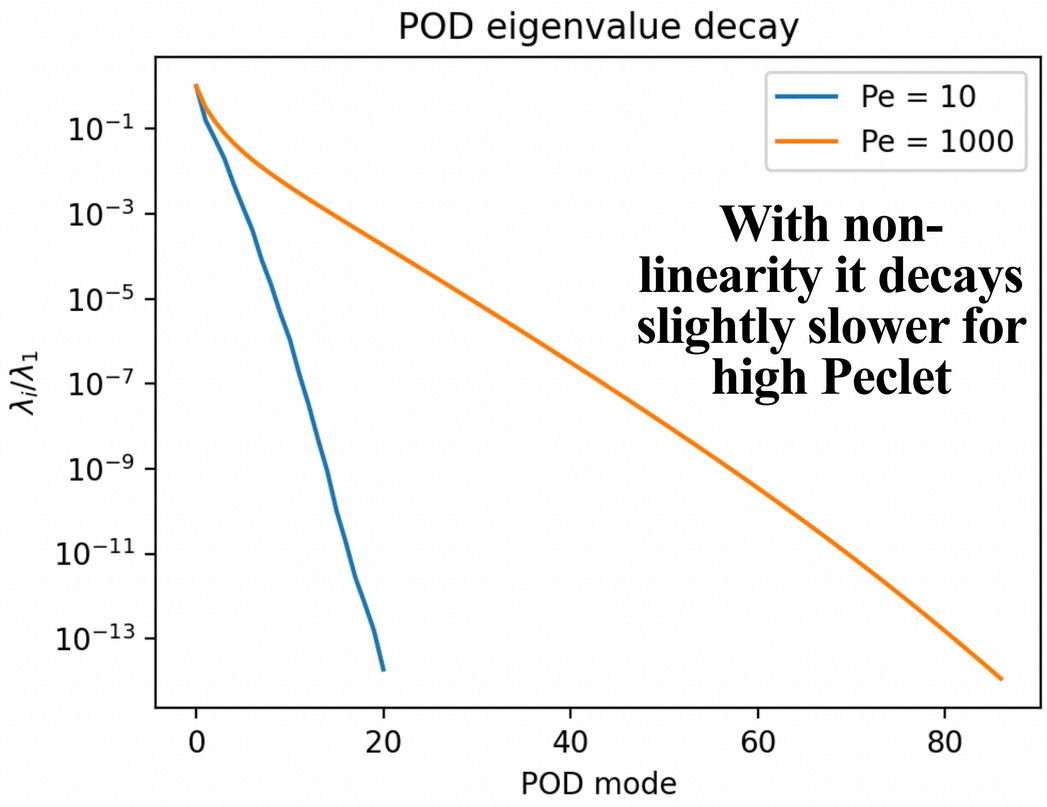


Reduced-Order Modelling

FEM for advection diffusion

We noticed as well some interesting type of dynamics when we add non-linearity:

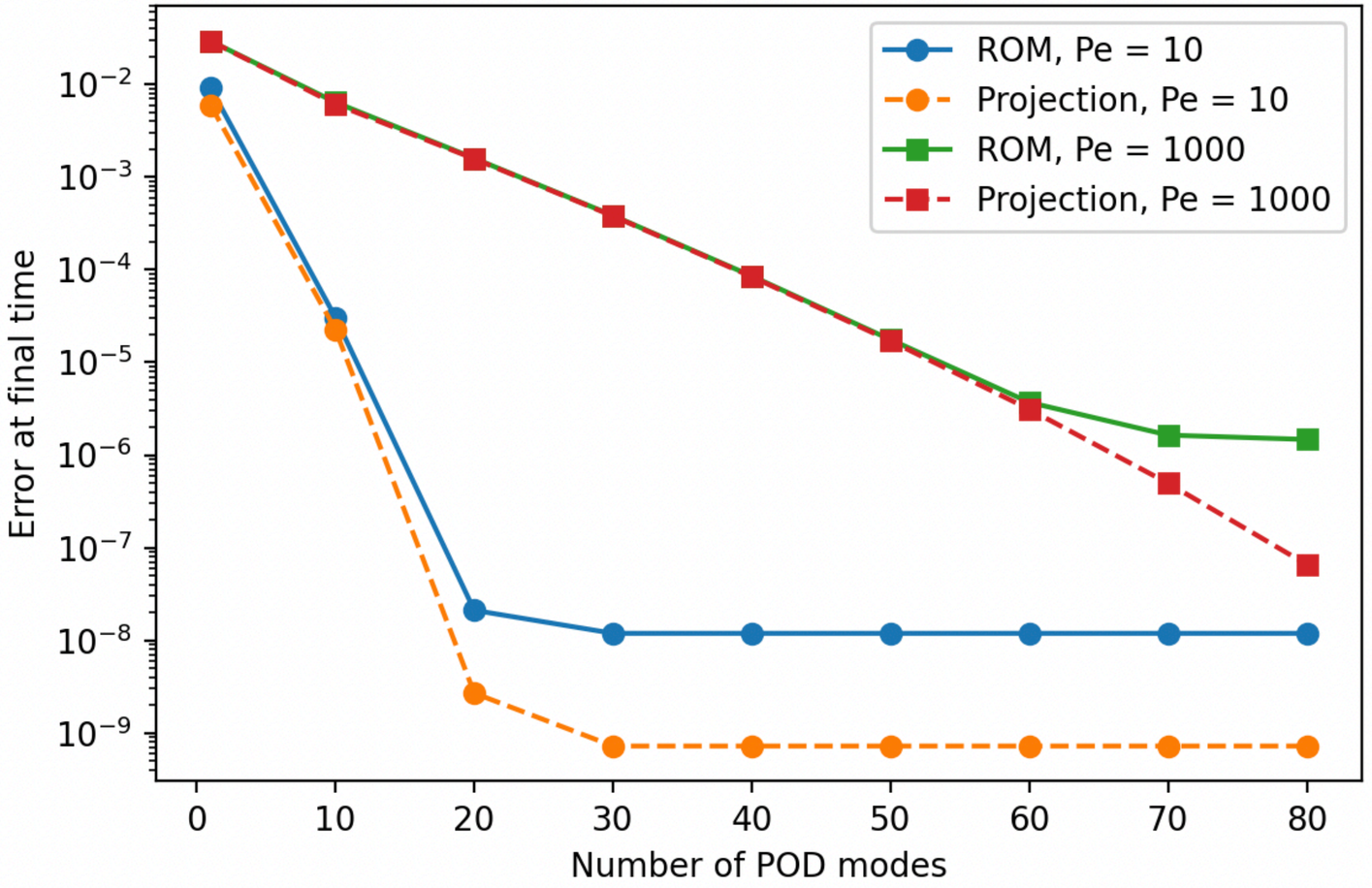




Reduced-Order Modelling

FEM for advection diffusion

Convergence analysis:



Some interpretation of the results:

1. As the number of modes retain increases you generally get smaller errors
2. The error goes down faster for smaller values of Pe (more viscous cases)
3. There are two different types of errors: projection errors and error in the dynamics of the ROM
4. All the errors at some point saturate
    1. Projection error saturates at 1e-8 simply cause I am working in single precision
    2. The ROM saturates because the error of the dynamics is dominant

Reduced-Order Modelling

FEM for advection diffusion

The whole point of ROMs, however, is about speed-ups. So, is the ROM faster than the FOM?

Full-order model (FE)

```
Pe = 1000: runtime = 182.220 s
```

Reduced-order model

```
Pe = 1000: 180.001 s
```

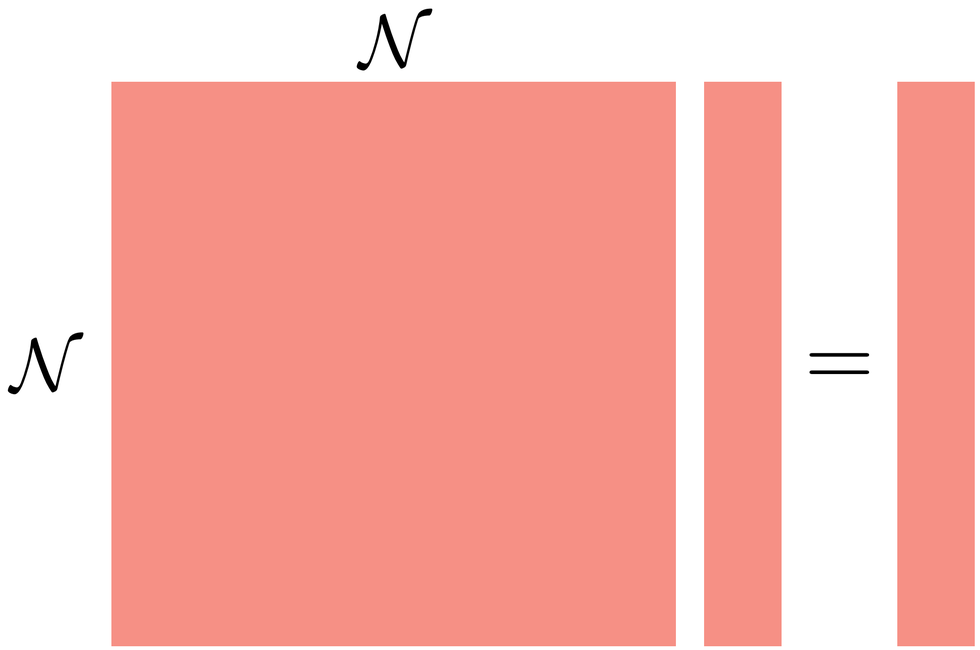
So, the ROM takes basically as much time as the FOM!! This is due to the naive way the non-linear term is treated. You end up with operations which scale with the size of the FOM and you lose any advantage.

Reduced-Order Modelling

FEM for advection diffusion

How do we solve this? We can maybe evaluate the non-linear terms not at all locations but only somewhere

$$\mathbf{Z}^T \mathbf{C}(\mathbf{Z}\mathbf{u}_N^t)\mathbf{Z}\mathbf{u}_N^t = \mathbf{Z}^T \mathbf{f}(\mathbf{u}_N^t)$$



The simple idea is to first "reduce" the non-linear term too and then to compute this term only at some locations. So, first, we look for a reduced representation of the type:

$$\mathbf{f}(\mathbf{u}_N^t) \approx \mathbf{G}\mathbf{c}$$

Where $\mathbf{G}$ represents a basis for the non-linear term and the target is to find the unknowns $\mathbf{c}$

Reduced-Order Modelling

FEM for advection diffusion

How do we solve this? We can maybe evaluate the non-linear terms not at all locations but only somewhere

You can easily guess how we want to find the optimal basis for the non-linear term. We can perform POD to the non-linear term itself:

$$\mathbf{F} = [\mathbf{f}^1, \mathbf{f}^2, ..., \mathbf{f}^{N_{max}}]$$

With

$$\mathbf{f}^t = \mathbf{C}(\mathbf{Z}\mathbf{u}_N^t)\mathbf{Z}\mathbf{u}_N^t$$

Via POD we get

$$\mathbf{G} = [\mathbf{g}^1, \mathbf{g}^2, ..., \mathbf{g}^m]$$

The simple idea is to first "reduce" the non-linear term too and then to compute this term only at some locations. So, first, we look for a reduced representation of the type:

$$\mathbf{f}(\mathbf{u}_N^t) \approx \mathbf{G}\mathbf{c}$$

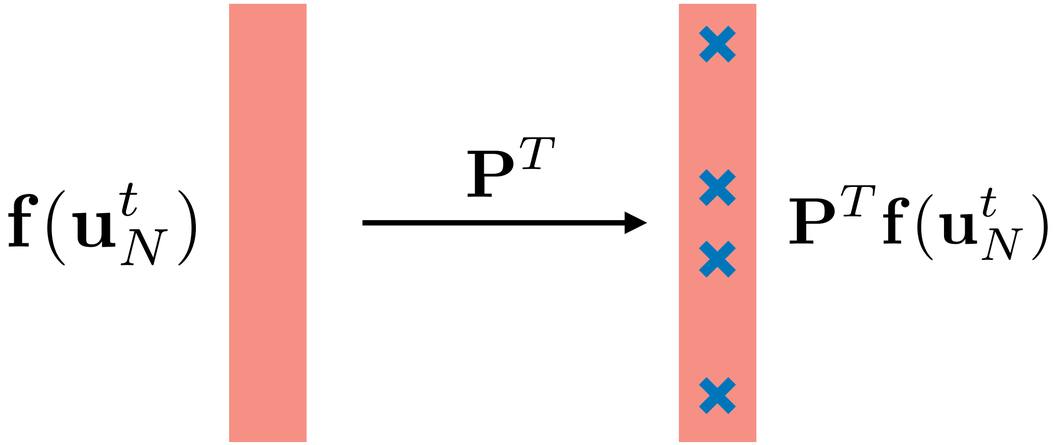
Where $\mathbf{G}$ represents a basis for the non-linear term and the target is to find the unknowns $\mathbf{c}$

Reduced-Order Modelling

FEM for advection diffusion

How do we solve this? We can maybe evaluate the non-linear terms not at all locations but only somewhere

How do we find $\mathbf{c}$ then? From a dimensional point of view we cannot solve the problem on the right: the number of unknowns is much smaller than the number of sampling. Then, we subsample!



The simple idea is to compute this term only at some locations:

$$\mathbf{f}(\mathbf{u}_N^t) \approx \mathbf{G}\mathbf{c}$$

Where $\mathbf{G}$ represents a basis for the non-linear term and the target is to find the unknowns $\mathbf{c}$

Reduced-Order Modelling

FEM for advection diffusion

How do we solve this? We can maybe evaluate the non-linear terms not at all locations but only somewhere

We can find $\mathbf{c}$ as

$$\mathbf{P}^T\mathbf{f}(\mathbf{u}_N^t) = \mathbf{P}^T\mathbf{G}\mathbf{c}$$

This linear system is well-posed and we can solve it

$$\mathbf{c} = (\mathbf{P}^T\mathbf{G})^{-1}\mathbf{P}^T\mathbf{f}(\mathbf{u}_N^t)$$

And get then the approximation:

$$\mathbf{f}(\mathbf{u}_N^t) \approx \mathbf{G}(\mathbf{P}^T\mathbf{G})^{-1}\mathbf{P}^T\mathbf{f}(\mathbf{u}_N^t)$$

The simple idea is to compute this term only at some locations:

$$\mathbf{f}(\mathbf{u}_N^t) \approx \mathbf{G}\mathbf{c}$$

Where $\mathbf{G}$ represents a basis for the non-linear term and the target is to find the unknowns $\mathbf{c}$

Reduced-Order Modelling

FEM for advection diffusion

$$\mathbf{f}(\mathbf{u}_N^t) \approx \mathbf{G}(\mathbf{P}^T\mathbf{G})^{-1}\mathbf{P}^T\mathbf{f}(\mathbf{u}_N^t)$$

Reduced-Order Modelling

FEM for advection diffusion

$$\mathbf{f}(\mathbf{u}_N^t) \approx \mathbf{G}(\mathbf{P}^T\mathbf{G})^{-1}\mathbf{P}^T\mathbf{f}(\mathbf{u}_N^t)$$

Now, back to our specific problem of the convective non linear term:

$$\mathbf{Z}^T\mathbf{C}(\mathbf{Z}\mathbf{u}_N^t)\mathbf{Z}\mathbf{u}_N^t = \mathbf{Z}^T\mathbf{f}(\mathbf{u}_N^t) \approx \underbrace{\mathbf{Z}^T\mathbf{G}(\mathbf{P}^T\mathbf{G})^{-1}}_{\mathbf{B}}\mathbf{P}^T\mathbf{f}(\mathbf{u}_N^t)$$

This term can be precomputed offline similarly to all the others

Reduced-Order Modelling

FEM for advection diffusion

$$\mathbf{f}(\mathbf{u}_N^t) \approx \mathbf{G}(\mathbf{P}^T\mathbf{G})^{-1}\mathbf{P}^T\mathbf{f}(\mathbf{u}_N^t)$$

This is basically the non-linear term evaluated only at some specific locations

Now, back to our specific problem of the convective non linear term:

$$\mathbf{Z}^T\mathbf{C}(\mathbf{Z}\mathbf{u}_N^t)\mathbf{Z}\mathbf{u}_N^t = \mathbf{Z}^T\mathbf{f}(\mathbf{u}_N^t) \approx \underbrace{\mathbf{Z}^T\mathbf{G}(\mathbf{P}^T\mathbf{G})^{-1}}_{\mathbf{B}}\mathbf{P}^T\mathbf{f}(\mathbf{u}_N^t)$$

This term can be precomputed offline similarly to all the others

Reduced-Order Modelling

FEM for advection diffusion

$$\mathbf{P}^T \mathbf{f}(\mathbf{u}_N^t)$$

Reduced-Order Modelling

FEM for advection diffusion

$\mathbf{P}^T\mathbf{f}(\mathbf{u}_N^t)$

The standard approach to compute this, of course, would be:

$$\mathbf{u}_N^t \xrightarrow{\text{Reconstruct}} \mathbf{Z}\mathbf{u}_N^t \xrightarrow{\text{Compute non-linear term}} \mathbf{C}(\mathbf{Z}\mathbf{u}_N^t)\mathbf{Z}\mathbf{u}_N^t \xrightarrow{\text{Subsample}} \mathbf{P}^T\mathbf{f}(\mathbf{u}_N^t)$$

Of course once again, we do not compute the full term and the restrict it to the lower number of points.
Instead we do:

$$\mathbf{u}_N^t \xrightarrow{\text{Reconstruct}} \mathbf{Z}_l\mathbf{u}_N^t \xrightarrow{\text{Compute non-linear term}} \mathbf{P}^T\mathbf{f}(\mathbf{u}_N^t)$$

Only at some points

Only at some points

Reduced-Order Modelling

FEM for advection diffusion

Offline

1. Perform POD on the solution (and you get $\mathbf{Z}$)
2. Perform POD on the non-linear term (and you get $\mathbf{G}$)
3. Pick m row via the matrix $\mathbf{P}$
4. Reduced operators $\mathbf{M}_N = \mathbf{Z}^T\mathbf{M}\mathbf{Z}$, $\mathbf{K}_N = \mathbf{Z}^T\mathbf{K}\mathbf{Z}$.
5. Interpolation map $\mathbf{B} = \mathbf{Z}^T\mathbf{G}(\mathbf{P}^T\mathbf{G})^{-1}$
6. Local data sample row $\mathbf{Z}_l$.

Online

1. Start with $\mathbf{u}_N^t$.
2. Subsample the reconstruction $\mathbf{v}_N^t = \mathbf{Z}_l\mathbf{u}_N^t$.
3. Compute the non-linear term only there:
   $\mathbf{P}^T\mathbf{f}(\mathbf{Z}\mathbf{u}_N^t) = \mathbf{f}(\mathbf{v}_N^t) = \mathbf{f}_p$.
4. Project the non-linear term: $\mathbf{s} = \mathbf{B}\mathbf{f}_p$
5. Update the solution:
   $\mathbf{u}_N^{t+1} = \mathbf{u}_N^t - \Delta t\mathbf{M}_N^{-1}(\mathbf{s} + \mathbf{K}_N\mathbf{u}_N^t/\mathrm{Pe})$.

Reduced-Order Modelling

FEM for advection diffusion

Offline

1. Perform POD on the solution (and you get $\mathbf{Z}$)
2. Perform POD on the non-linear term (and you get $\mathbf{G}$)
3. Pick m row via the matrix $\mathbf{P}$
4. Reduced operators $\mathbf{M}_N = \mathbf{Z}^T\mathbf{M}\mathbf{Z}$, $\mathbf{K}_N = \mathbf{Z}^T\mathbf{K}\mathbf{Z}$.
5. Interpolation map $\mathbf{B} = \mathbf{Z}^T\mathbf{G}(\mathbf{P}^T\mathbf{G})^{-1}$
6. Local data sample row $\mathbf{Z}_l$.

Online

1. Start with $\mathbf{u}_N^t$.	
2. Subsample the reconstruction $\mathbf{v}_N^t = \mathbf{Z}_l\mathbf{u}_N^t$.	$\mathcal{O}(m \cdot N)$
3. Compute the non-linear term only there: $\mathbf{P}^T\mathbf{f}(\mathbf{Z}\mathbf{u}_N^t) = \mathbf{f}(\mathbf{v}_N^t) = \mathbf{f}_p$.	$\mathcal{O}(m)$
4. Project the non-linear term: $\mathbf{s} = \mathbf{B}\mathbf{f}_p$	$\mathcal{O}(m \cdot N)$
5. Update the solution: $\mathbf{u}_N^{t+1} = \mathbf{u}_N^t - \Delta t\mathbf{M}_N^{-1}(\mathbf{s} + \mathbf{K}_N\mathbf{u}_N^t/\mathrm{Pe})$.	$\mathcal{O}(N^2)$

Reduced-Order Modelling

FEM for advection diffusion

We solve:

$$\mathbf{Z}^T\mathbf{M}\mathbf{Z}\frac{\mathbf{u}_N^{t+1}-\mathbf{u}_N^t}{\Delta t}+\mathbf{Z}^T\mathbf{C}(\mathbf{Z}\mathbf{u}_N^t)\mathbf{Z}\mathbf{u}_N^t=-\frac{1}{\mathrm{Pe}}\mathbf{Z}^T\mathbf{K}\mathbf{Z}\mathbf{u}_N^t.$$

This was the naive first implementation of the ROM for the non-linear case

```
def rom(Z, S, Pe):
    a = Z.T @ M @ S[:, 0]
    K_N = Z.T @ K @ Z
    start = time.perf_counter()
    for _ in range(n_steps):
        u = Z @ a
        a -= dt * (K_N @ a / Pe + Z.T @ (C @ (u * u)))

    return Z @ a, time.perf_counter() - start
```

Reduced-Order Modelling

FEM for advection diffusion

We solve:

$$\mathbf{Z}^T\mathbf{M}\mathbf{Z}\frac{\mathbf{u}_N^{t+1}-\mathbf{u}_N^t}{\Delta t}+\mathbf{Z}^T\mathbf{C}(\mathbf{Z}\mathbf{u}_N^t)\mathbf{Z}\mathbf{u}_N^t=-\frac{1}{\mathrm{Pe}}\mathbf{Z}^T\mathbf{K}\mathbf{Z}\mathbf{u}_N^t.$$

So, instead of computing the non-linear term naively we will need some extra work both offline and online

```
def rom_deim(Z, S, Pe, G, p):
    B,C_P,Z_S = deim_setup(Z,G,p)
    a = Z.T @ M @ S[:, 0]
    K_N = Z.T @ K @ Z
    start = time.perf_counter()
    for _ in range(n_steps):
        u_S = Z_S @ a
        f_P = C_P @ (u_S * u_S)
        a -= dt * (K_N @ a / Pe + B @ f_P)

    return Z @ a, time.perf_counter() - start
```

Reduced-Order Modelling

FEM for advection diffusion

We solve:

$$\mathbf{Z}^T\mathbf{M}\mathbf{Z}\frac{\mathbf{u}_N^{t+1}-\mathbf{u}_N^t}{\Delta t}+\mathbf{Z}^T\mathbf{C}(\mathbf{Z}\mathbf{u}_N^t)\mathbf{Z}\mathbf{u}_N^t=-\frac{1}{\mathrm{Pe}}\mathbf{Z}^T\mathbf{K}\mathbf{Z}\mathbf{u}_N^t.$$

So, instead of computing the non-linear term naively we will need some extra work both offline and online

```
def rom_deim(Z, S, Pe, G, p):
    B,C_P,Z_S = deim_setup(Z,G,p)
    a = Z.T @ M @ S[:, 0]
    K_N = Z.T @ K @ Z
    start = time.perf_counter()
    for _ in range(n_steps):
        u_S = Z_S @ a
        f_P = C_P @ (u_S * u_S)
        a -= dt * (K_N @ a / Pe + B @ f_P)

    return Z @ a, time.perf_counter() - start
```

Reduced-Order Modelling

FEM for advection diffusion

The main objective of the DEIM is to reduce the computational cost of the the non-linear term without loosing accuracy. From our numerical experiments we notice that we can obtain a pretty nice speed-up (as expected)

```
ROM runtime:       Pe = 10: 181.631 s, Pe = 1000: 170.268 s
DEIM-ROM runtime:  Pe = 10: 4.889 s, Pe = 1000: 5.335 s
Speed-up:          Pe = 10: 37.2x, Pe = 1000: 31.9x
```

However….

```
Pe = 10:   final-time error ROM = 2.531221e-04, DEIM-ROM = 2.737297e-04
Pe = 1000: final-time error ROM = 7.889092e-04, DEIM-ROM = 8.752288e-03
```

Reduced-Order Modelling

FEM for advection diffusion

However….

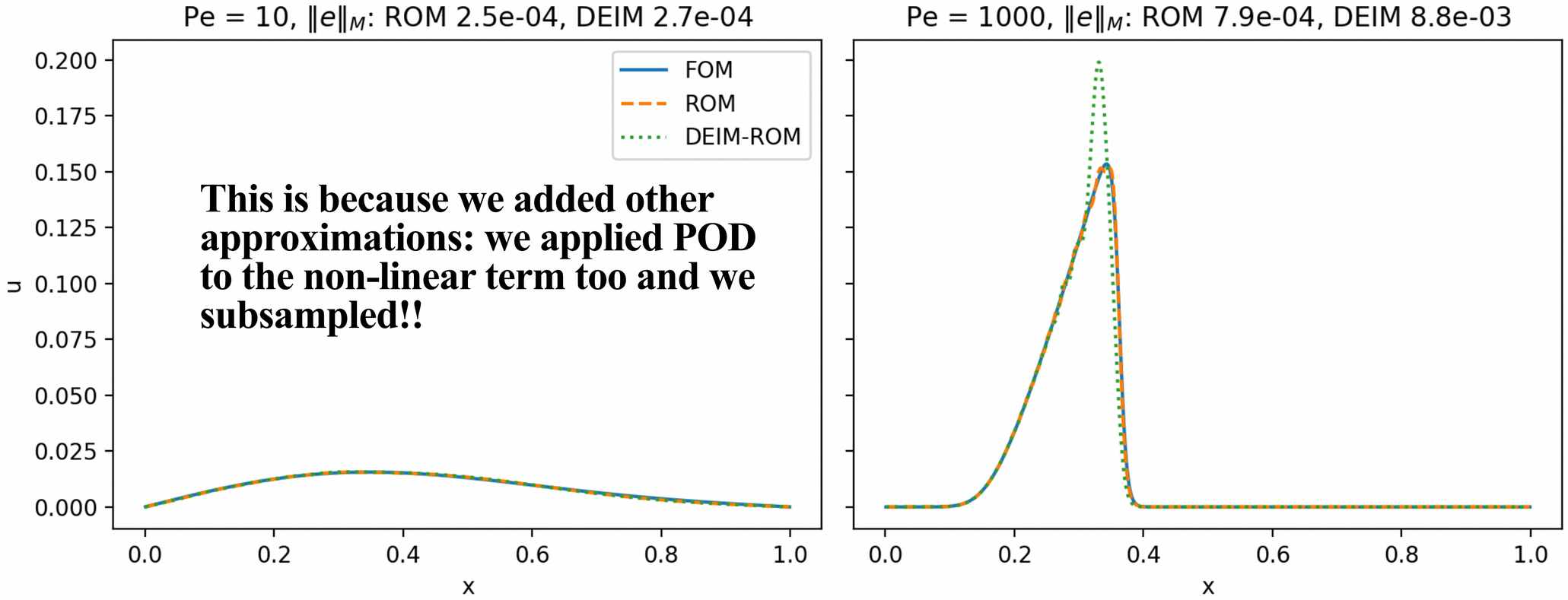
```
Pe = 10:   final-time error ROM = 2.531221e-04, DEIM-ROM = 2.737297e-04
Pe = 1000: final-time error ROM = 7.889092e-04, DEIM-ROM = 8.752288e-03
```

Reduced-Order Modelling

FEM for advection diffusion

However….

```
Pe = 10:   final-time error ROM = 2.531221e-04, DEIM-ROM = 2.737297e-04
Pe = 1000: final-time error ROM = 7.889092e-04, DEIM-ROM = 8.752288e-03
```

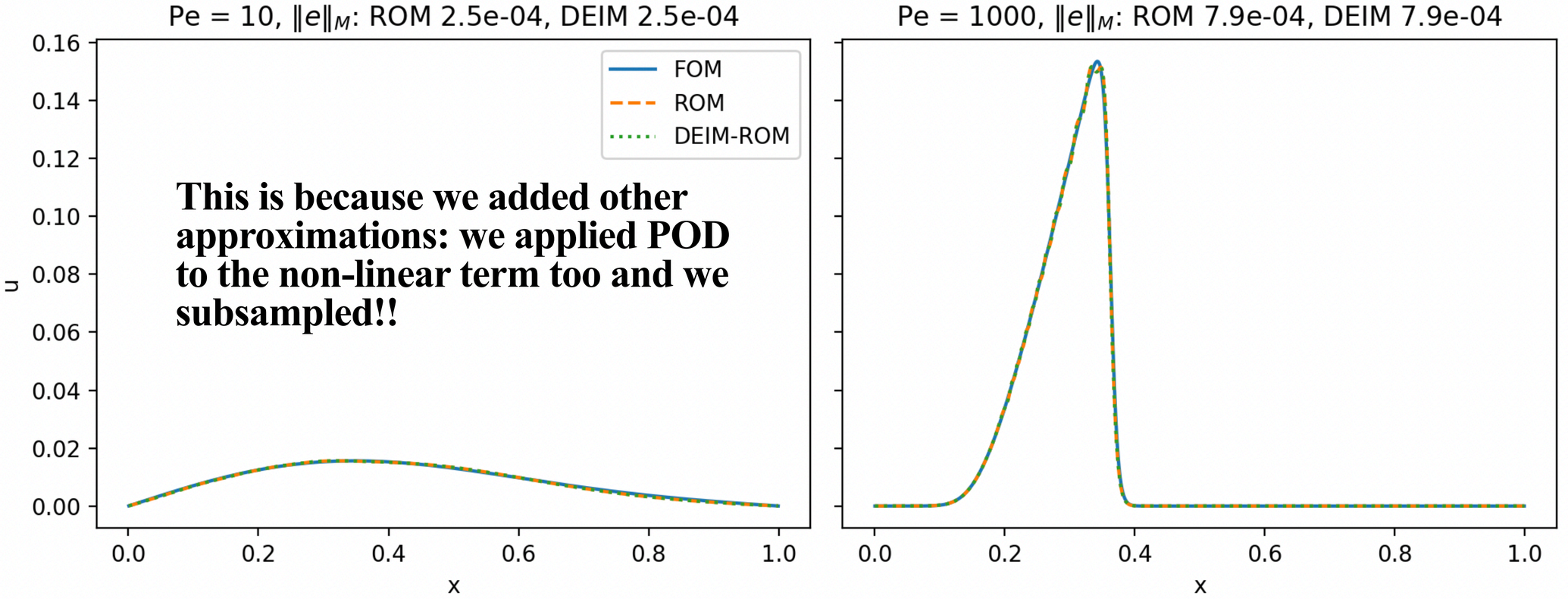


Reduced-Order Modelling

FEM for advection diffusion

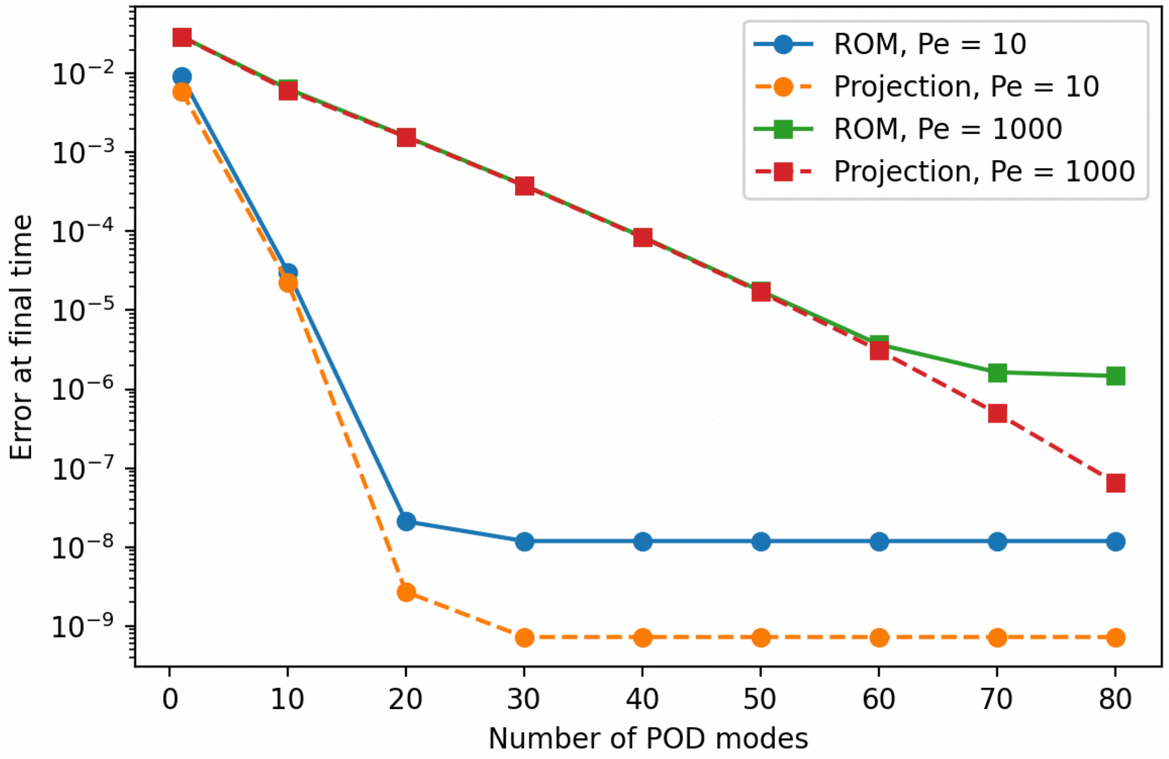
However….

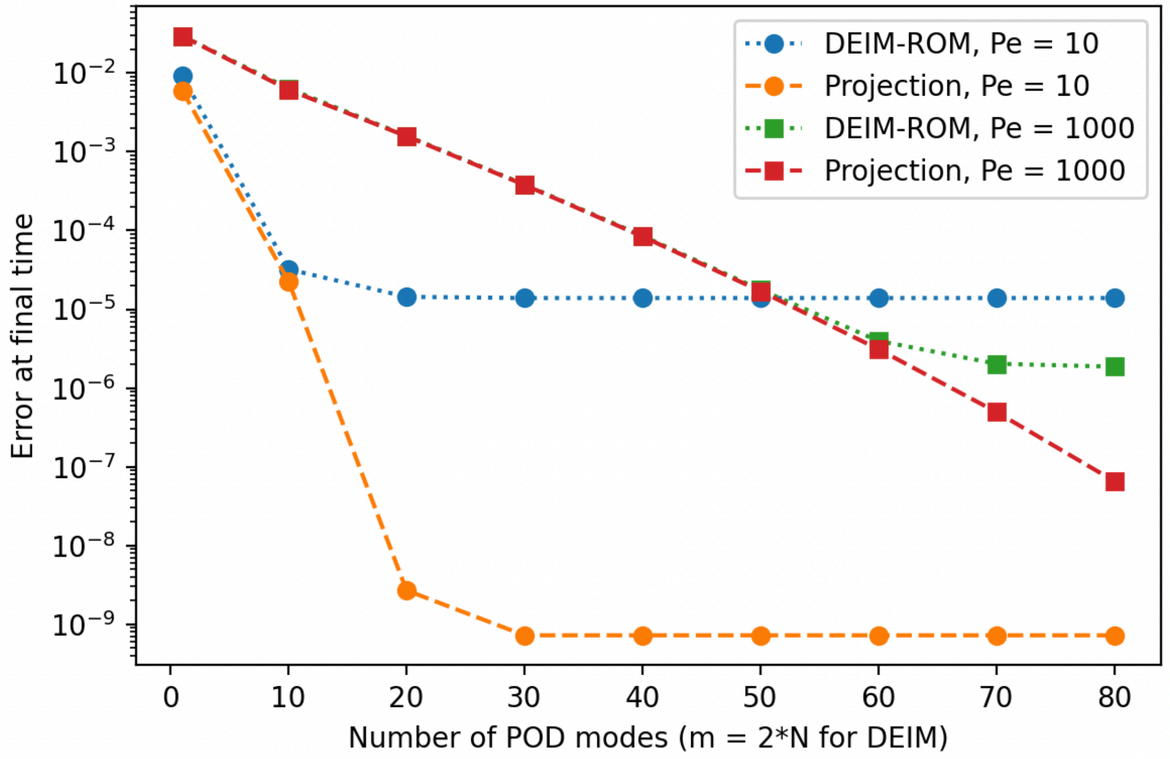
```
Pe = 10:   final-time error ROM = 2.531221e-04, DEIM-ROM = 2.535574e-04
Pe = 1000: final-time error ROM = 7.889092e-04, DEIM-ROM = 7.889930e-04
```



Reduced-Order Modelling

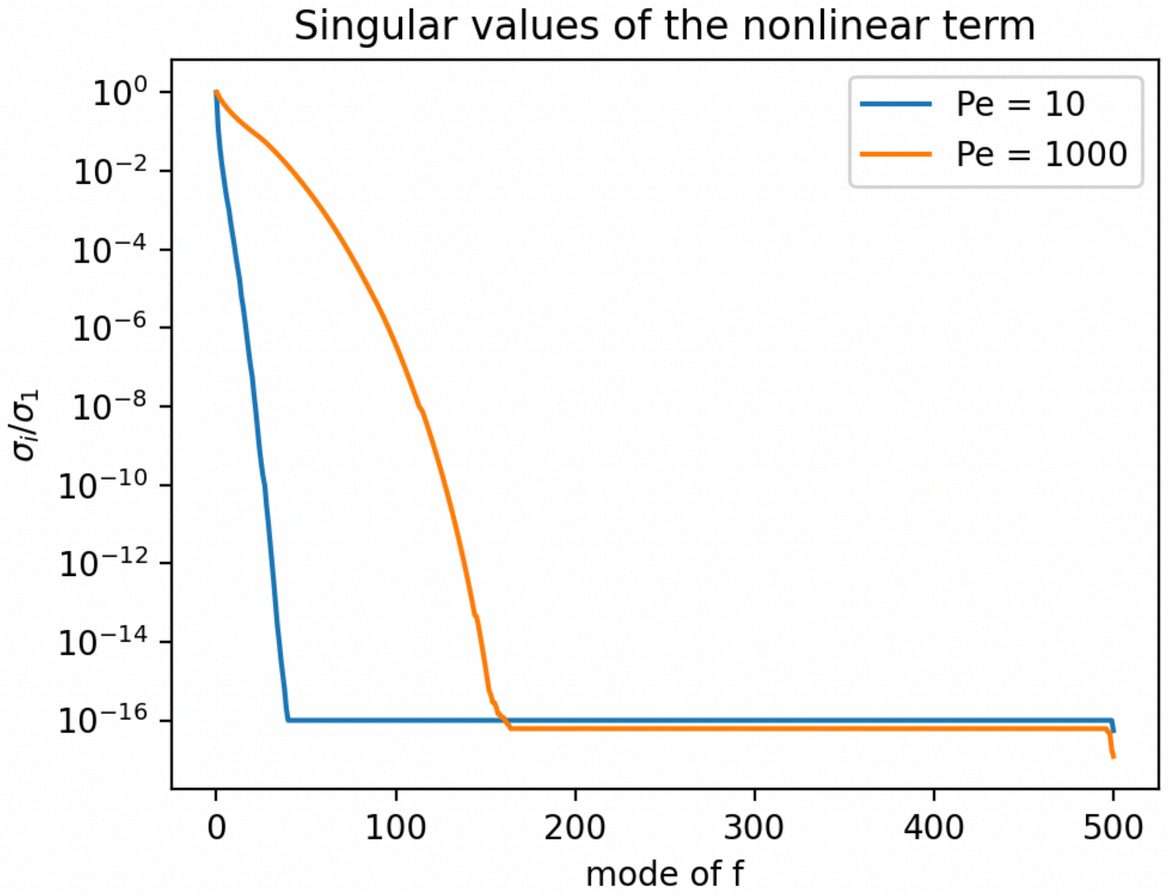
FEM for advection diffusion

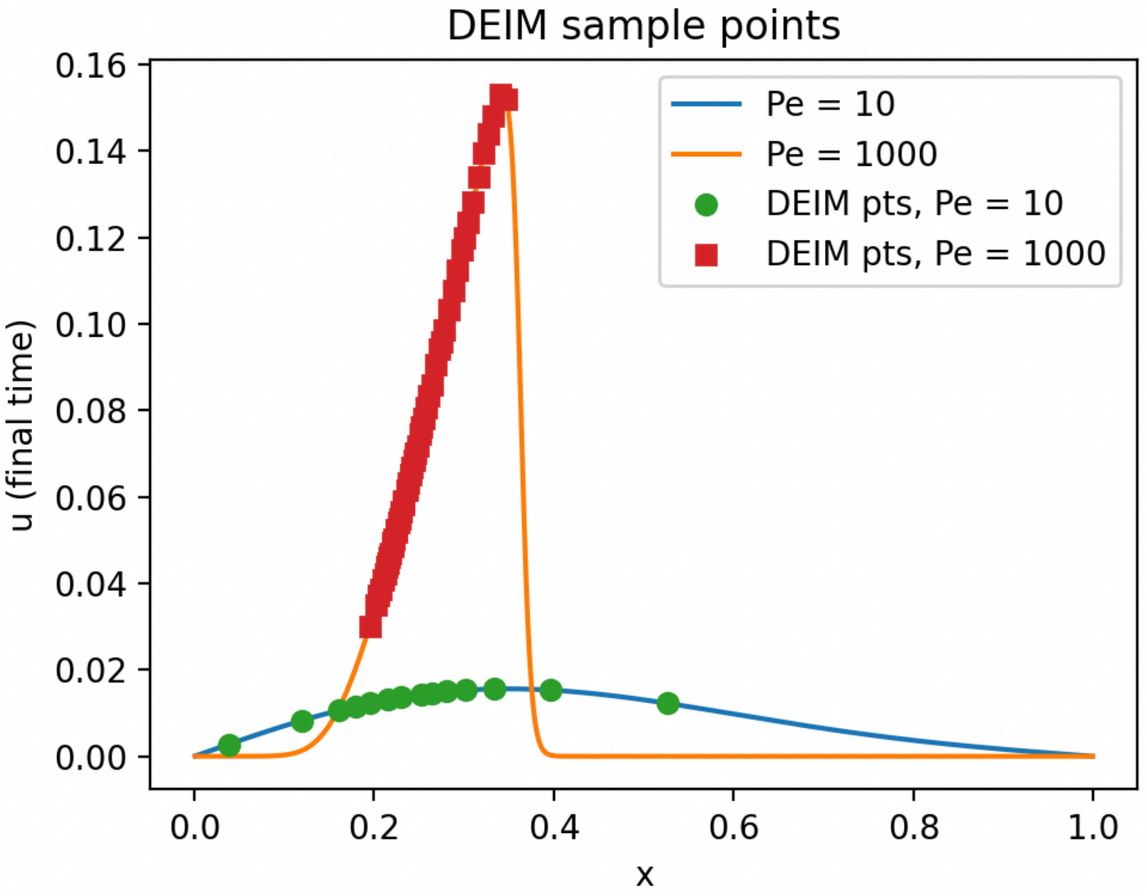




Reduced-Order Modelling

FEM for advection diffusion





Reduced-Order Modelling

ROM for Navier-Stokes

Parametrised incompressible Navier-Stokes, with parameter μ = ν (or the Reynolds number):

$$\frac{\partial u_i}{\partial t} + u_j \frac{\partial u_i}{\partial x_j} = \nu \frac{\partial^2 u_i}{\partial x_j \partial x_j} - \frac{\partial p}{\partial x_i}$$
$$\frac{\partial u_i}{\partial x_i} = 0$$

After a finite element or finite volume discretisation with N_h degrees of freedom:

$$\mathbf{M}\frac{d\mathbf{u}}{dt} + \nu\mathbf{K}\mathbf{u} + \mathbf{C}(\mathbf{u})\mathbf{u} + \mathbf{B}^T\mathbf{p} = 0$$
$$\mathbf{B}\mathbf{u} = 0$$

What the ROM has to handle

- **Non-linearity:** the convective term C(u) u is quadratic in u
- **Saddle point:** B couples velocity and pressure
- **Parameter:** ν enters linearly, Re = UL/ν
- **Size:** N_h of 10^5 to 10^7 and many time steps

Reduced-Order Modelling

ROM for Navier-Stokes

$$\mathbf{M}\frac{d\mathbf{u}}{dt} + \nu\mathbf{K}\mathbf{u} + \mathbf{C}(\mathbf{u})\mathbf{u} + \mathbf{B}^T\mathbf{p} = 0$$

$$\mathbf{B}\mathbf{u} = 0$$

Reduced-Order Modelling

ROM for Navier-Stokes

$$\mathbf{M}\frac{d\mathbf{u}}{dt} + \nu\mathbf{K}\mathbf{u} + \mathbf{C}(\mathbf{u})\mathbf{u} + \mathbf{B}^T\mathbf{p} = 0$$

$$\mathbf{B}\mathbf{u} = 0$$

The structure of the problem is very similar to the one we considered in the previous slides with the addition of the pressure variable (which everything but trivial).

The pipeline for an intrusive ROM for Navier-Stokes is still very similar as the standard approach. We will apply POD to both velocity and pressure variables:

Velocity

$$\mathbf{u}^N = \mathbf{Z}_u\mathbf{u}^{\mathcal{N}}$$

Pressure

$$\mathbf{p}^N = \mathbf{Z}_p\mathbf{p}^{\mathcal{N}}$$

Then we can project the equations as before:

$$\mathbf{Z}_u\mathbf{M}\mathbf{Z}_u^T\frac{d\mathbf{u}^N}{dt} + \nu\mathbf{Z}_u\mathbf{K}\mathbf{Z}_u^T\mathbf{u}^N + \mathbf{Z}_u\mathbf{C}(\mathbf{u})\mathbf{Z}_u^T\mathbf{u}^N + \mathbf{Z}_u\mathbf{B}^T\mathbf{Z}_p\mathbf{p}_N = 0$$

$$\mathbf{Z}_p\mathbf{B}\mathbf{Z}_u^T\mathbf{u}^N = 0$$

Reduced-Order Modelling

ROM for Navier-Stokes

$$\mathbf{Z}_u \mathbf{M} \mathbf{Z}_u^T \frac{d\mathbf{u}^N}{dt} + \nu \mathbf{Z}_u \mathbf{K} \mathbf{Z}_u^T \mathbf{u}^N + \mathbf{Z}_u \mathbf{C}(\mathbf{u}) \mathbf{Z}_u^T \mathbf{u}^N + \mathbf{Z}_u \mathbf{B}^T \mathbf{Z}_p \mathbf{p}_N = 0$$

$$\mathbf{Z}_p \mathbf{B} \mathbf{Z}_u^T \mathbf{u}^N = 0$$

Reduced-Order Modelling

FEM for Navier-Stokes

$$\mathbf{Z}_u \mathbf{M} \mathbf{Z}_u^T \frac{d\mathbf{u}^N}{dt} + \nu \mathbf{Z}_u \mathbf{K} \mathbf{Z}_u^T \mathbf{u}^N + \mathbf{Z}_u \mathbf{C}(\mathbf{u}) \mathbf{Z}_u^T \mathbf{u}^N + \mathbf{Z}_u \mathbf{B}^T \mathbf{Z}_p \mathbf{p}_N = 0$$

$$\mathbf{Z}_p \mathbf{B} \mathbf{Z}_u^T \mathbf{u}^N = 0$$

POD-Galerkin for Navier-Stokes: challenges

Supremizer enrichment [1, 2, 5]

- Velocity and pressure both stay in the reduced system
- Add supremizers to the velocity space restore the inf-sup condition
- Cost: a larger velocity basis

Pressure Poisson equation [3, 4, 5]

- Reduced momentum equation for the velocity only
- Pressure recovered from the velocity with $\Delta p = -\nabla\cdot((u\cdot\nabla)u)$: no inf-sup constraint
- Accuracy depends on the pressure boundary conditions

Instabilities

Truncation removes the dissipative small scales, so energy builds up and long-time solutions can drift or blow up, especially at high Re [3, 7].

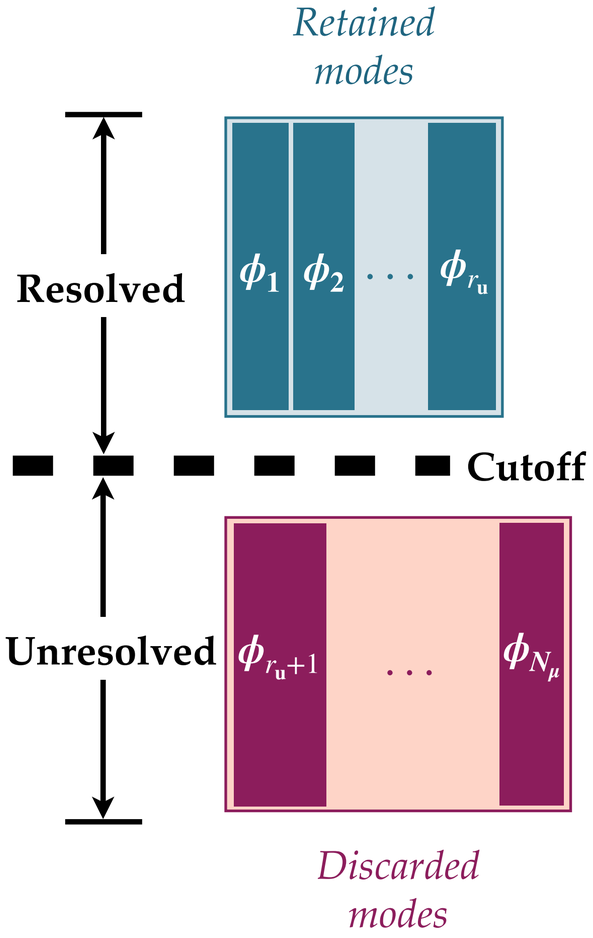
Stabilisation

- Eddy or spectral viscosity added to ν [6]
- Models with long-term boundedness [7]
- Least-squares Petrov-Galerkin projection [8]
- Data-driven closure terms

[1] Rozza, Veroy, CMAME (2007)
[2] Ballarin, Manzoni, Quarteroni, Rozza, IJNME (2015)
[3] Noack, Papas, Monkewitz, JFM (2005)
[4] Caiazzo, Iliescu, John, Schyschlowa, JCP (2014)
[5] Stabile, Rozza, Computers & Fluids (2018)
[6] Sirisup, Karniadakis, JCP (2004)
[7] Schlegel, Noack, JFM (2015)
[8] Carlberg, Bou-Mosleh, Farhat, IJNME (2011)

Reduced-Order Modelling

ROM for Navier-Stokes



This is an example from Ivagnes et al. where the so called variational multiscale closure method is considered:

$$\mathbf{M}_N \frac{d\mathbf{u}^N}{dt} + \nu \mathbf{K}_N \mathbf{u}^N + \widetilde{\mathbf{C}(\mathbf{u})}\mathbf{u}^N + \mathbf{B}_N^T \mathbf{p}^N + \tau = 0$$

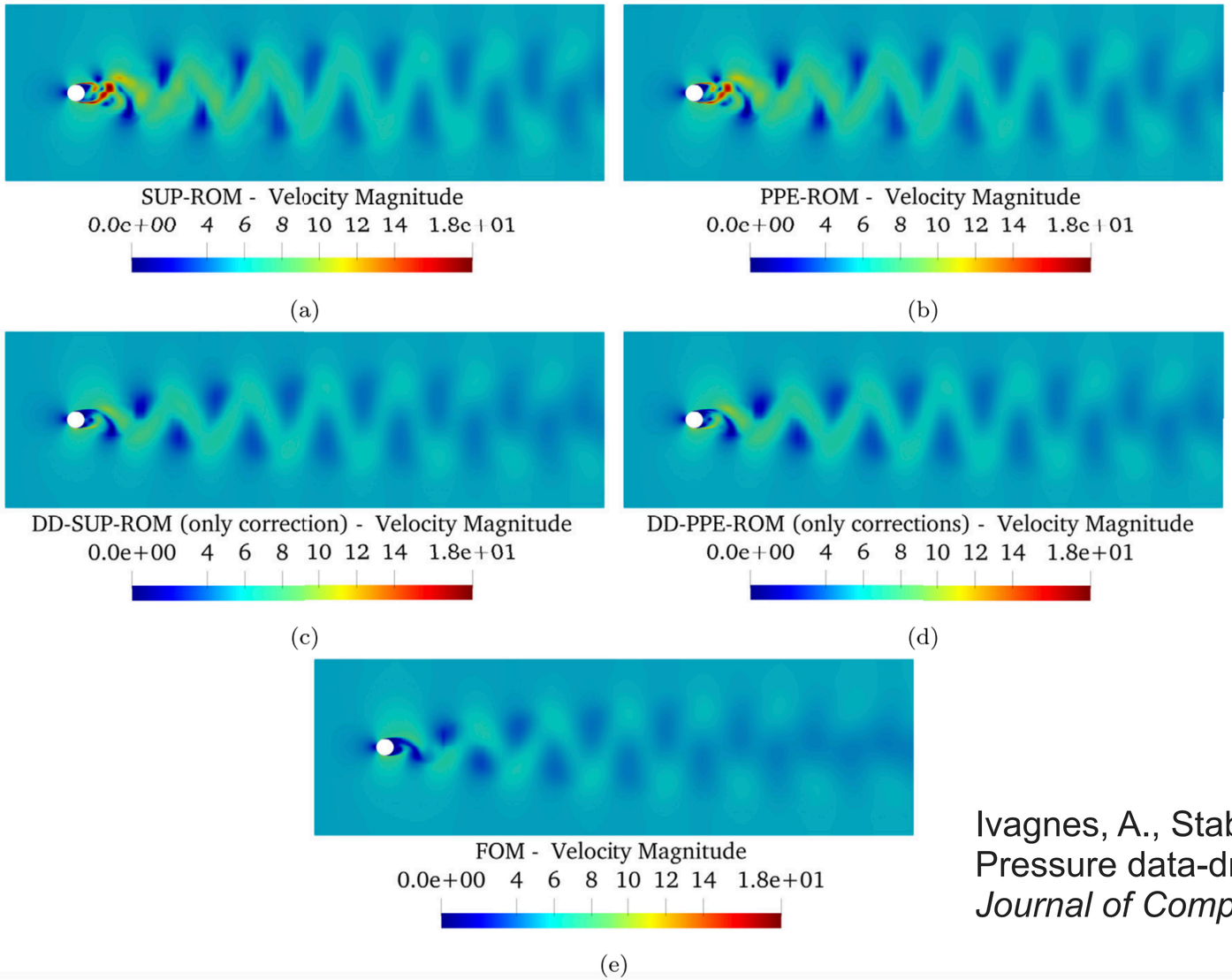
$$\mathbf{B}_N \mathbf{u}^N = 0$$

Where τ represents all the "errors" we are committing by neglecting the higher modes.

Ivagnes, A., Stabile, G., Mola, A., Iliescu, T., & Rozza, G. (2023). Pressure data-driven variational multiscale reduced order models. *Journal of Computational Physics*, *476*, 111904.

Reduced-Order Modelling

ROM for Navier-Stokes



This is an example from Ivagnes et al. where the so called variational multiscale closure method is considered:

$$\mathbf{M}_N \frac{d\mathbf{u}^N}{dt} + \nu \mathbf{K}_N \mathbf{u}^N + \widetilde{\mathbf{C}(\mathbf{u})}\mathbf{u}^N + \mathbf{B}_N^T \mathbf{p}^N + \tau = 0$$

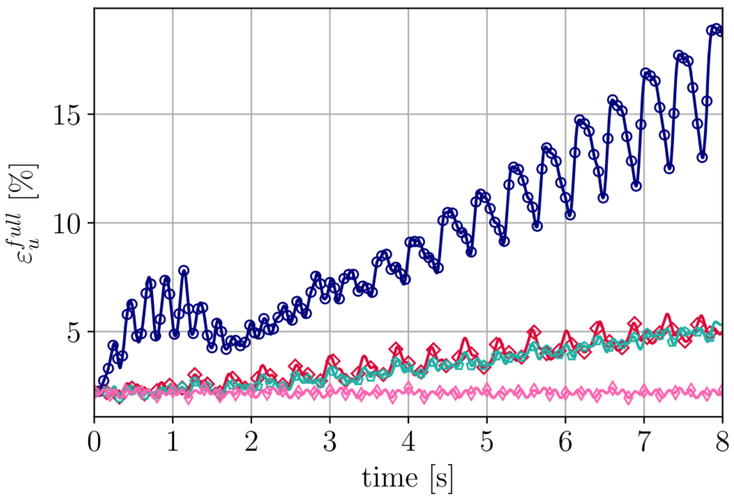
$$\mathbf{B}_N \mathbf{u}^N = 0$$

Where τ represents all the "errors" we are committing by neglecting the higher modes.

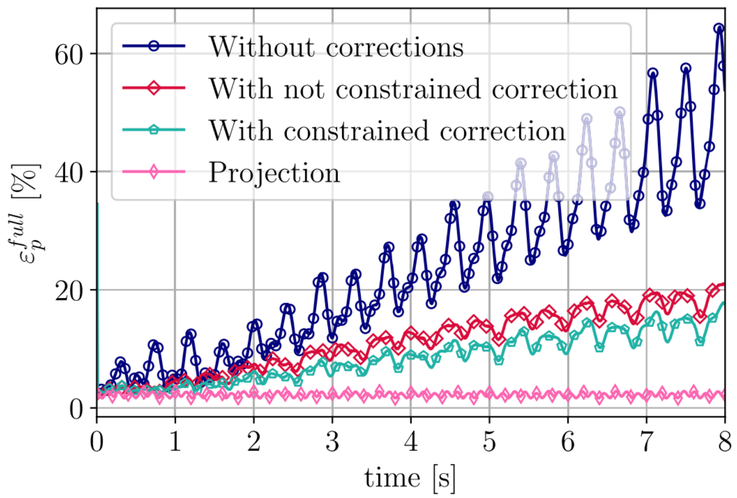
Ivagnes, A., Stabile, G., Mola, A., Iliescu, T., & Rozza, G. (2023). Pressure data-driven variational multiscale reduced order models. *Journal of Computational Physics*, *476*, 111904.

Reduced-Order Modelling

ROM for Navier-Stokes



(a) Relative error on velocity



(b) Relative error on pressure

This is an example from Ivagnes et al. where the so called variational multiscale closure method is considered:

$$\mathbf{M}_N \frac{d\mathbf{u}^N}{dt} + \nu \mathbf{K}_N \mathbf{u}^N + \widetilde{\mathbf{C}(\mathbf{u})}\mathbf{u}^N + \mathbf{B}_N^T \mathbf{p}^N + \tau = 0$$

$$\mathbf{B}_N \mathbf{u}^N = 0$$

Where τ represents all the "errors" we are committing by neglecting the higher modes.

Ivagnes, A., Stabile, G., Mola, A., Iliescu, T., & Rozza, G. (2023). Pressure data-driven variational multiscale reduced order models. *Journal of Computational Physics*, *476*, 111904.

Reduced-Order Modelling

A little bit of recap on intrusive ROMs

An intrusive ROM takes the discretised governing equations and projects them onto a reduced basis, so the ROM is a small version of the same model. You can achieve that by precomputing many low-dimensional operators in the offline phase and predict a small number of coefficients in the online phase.

Pros

1. They are physics-based: they actually use the governing equations and their discretised version
    1. Little data needed
    2. Good extrapolation
    3. Good interpretability
2. Error control: well established field with solid math

Cons

1. They are physics-based: they actually use the governing equations and their discretised version
    1. You need to know the details of your FOM
    2. Hyper-reduction for non-linear terms
2. Stability issues

Reduced-Order Modelling

Non-intrusive ROMs

Can we extract all the information we need from snapshots only? Can we avoid to use the projected equations to evolve the dynamics of the system?

Intrusive ROMs are clearly based on:

The computation of the POD basis from the snapshots matrix. We find the matrix $\mathbf{Z}$ such that we can compress the information of the full-order model into a small number of coefficients:

$$\mathbf{u}^{\mathcal{N}} \xrightarrow{\quad\mathbf{Z}\quad} \mathbf{u}^{N}$$

Step 1

The evolution of the POD coefficients $\mathbf{u}^{N}$ is then obtain from the projected equations

Step 2

$$\mathbf{Z}^T\mathbf{M}\mathbf{Z}\frac{\mathbf{u}_N^{t+1}-\mathbf{u}_N^t}{\Delta t}+\mathbf{Z}^T\mathbf{C}(\mathbf{Z}\mathbf{u}_N^t)\mathbf{Z}\mathbf{u}_N^t=-\frac{1}{\mathrm{Pe}}\mathbf{Z}^T\mathbf{K}\mathbf{Z}\mathbf{u}_N^t.$$

Reduced-Order Modelling

Non-intrusive ROMs

Can we extract all the information we need from snapshots only? Can we avoid to use the projected equations to evolve the dynamics of the system?

Intrusive ROMs are clearly based on:

The computation of the POD basis from the snapshots matrix. We find the matrix $\mathbf{Z}$ such that we can compress the information of the full-order model into a small number of coefficients:

$$\mathbf{u}^{\mathcal{N}} \xrightarrow{\quad\mathbf{Z}\quad} \mathbf{u}^{N}$$

Step 1

The evolution of the POD coefficients $\mathbf{u}^{N}$ is then obtain from the projected equations

Step 2

$$\mathbf{Z}^T\mathbf{M}\mathbf{Z}\frac{\mathbf{u}_N^{t+1}-\mathbf{u}_N^t}{\Delta t}+\mathbf{Z}^T\mathbf{C}(\mathbf{Z}\mathbf{u}_N^t)\mathbf{Z}\mathbf{u}_N^t=-\frac{1}{\mathrm{Pe}}\mathbf{Z}^T\mathbf{K}\mathbf{Z}\mathbf{u}_N^t.$$

Reduced-Order Modelling

Non-intrusive ROMs

Can we extract all the information we need from snapshots only? Can we avoid to use the projected equations to evolve the dynamics of the system?

Intrusive ROMs are clearly based on:

The computation of the POD basis from the snapshots matrix. We find the matrix $\mathbf{Z}$ such that we can compress the information of the full-order model into a small number of coefficients:

$$\mathbf{u}^{\mathcal{N}} \xrightarrow{\quad\mathbf{Z}\quad} \mathbf{u}^{N}$$

Step 1

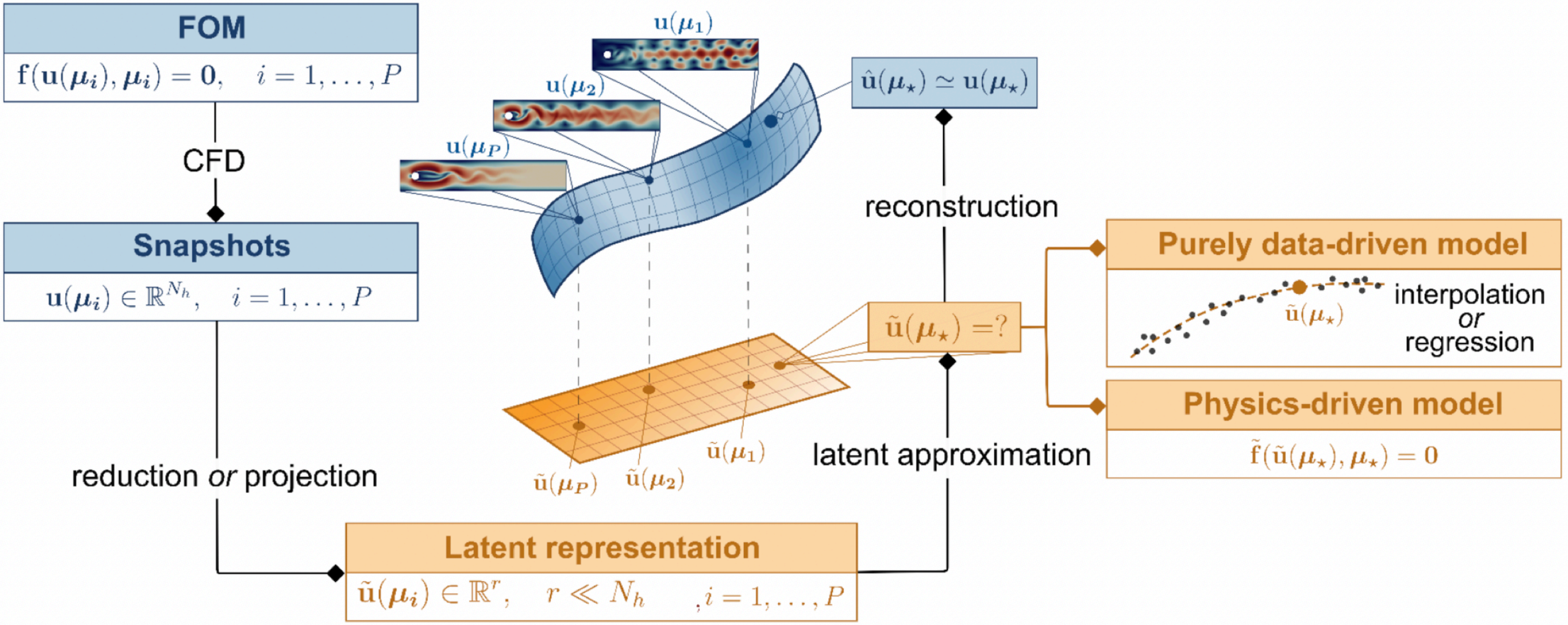
Now that we have a reduced basis we can project all the snapshots into this linear subspace:

$$\mathbf{a}_i = \mathbf{Z}^T \mathbf{M}\, \mathbf{u}(\boldsymbol{\mu}_i) \in \mathbb{R}^r, \qquad (\mathbf{a}_i)_k = \left(\boldsymbol{\zeta}_k,\ \mathbf{u}(\boldsymbol{\mu}_i)\right)_{\mathbf{M}}, \qquad i = 1, \ldots, n_{train}$$

$$\mathcal{T} = \left\{(\boldsymbol{\mu}_i,\ \mathbf{a}_i)\right\}_{i=1}^{n}, \qquad \boldsymbol{\mu} \longmapsto \mathbf{a}(\boldsymbol{\mu}) \approx \mathbf{Z}^T \mathbf{M}\, \mathbf{u}(\boldsymbol{\mu}).$$

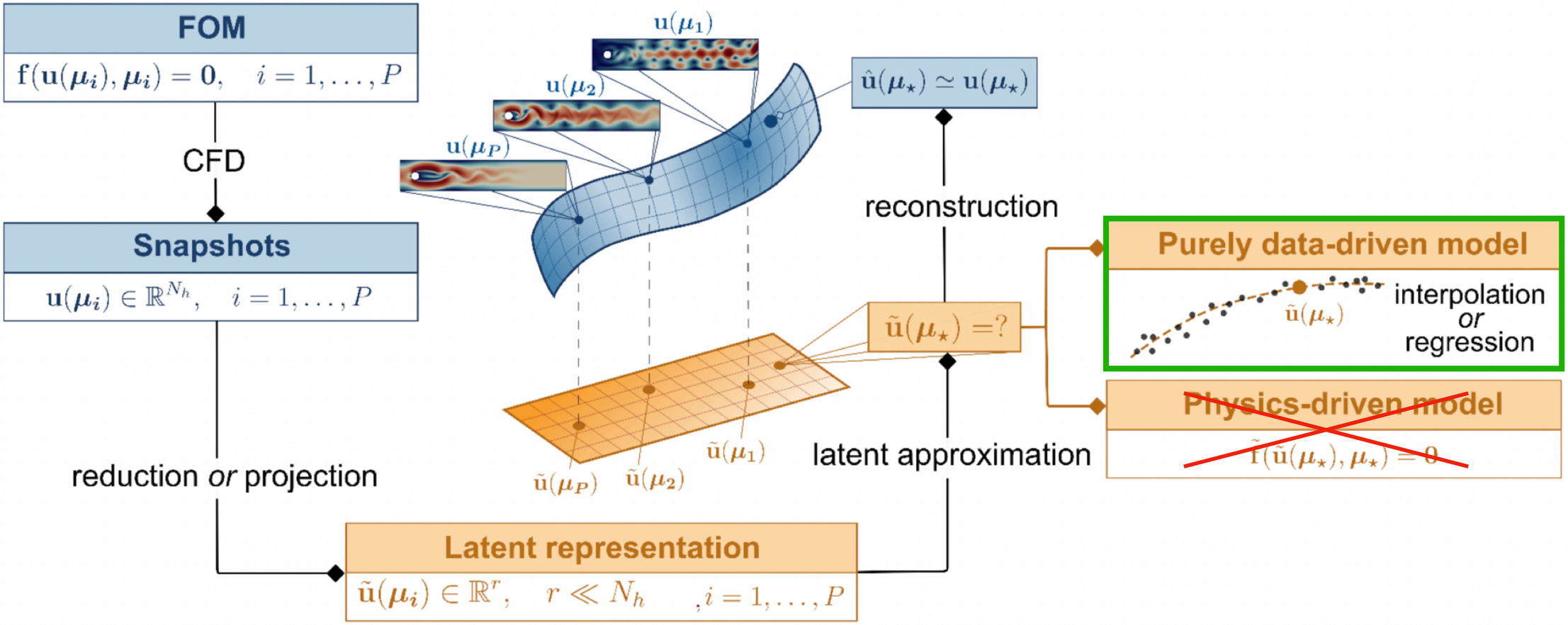
Reduced-Order Modelling

Non-intrusive ROMs



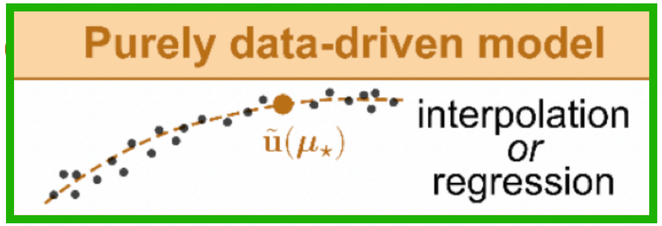
Reduced-Order Modelling

Non-intrusive ROMs



Reduced-Order Modelling

Non-intrusive ROMs



Reduced-Order Modelling

Non-intrusive ROMs

Purely data-driven model



interpolation *or* regression

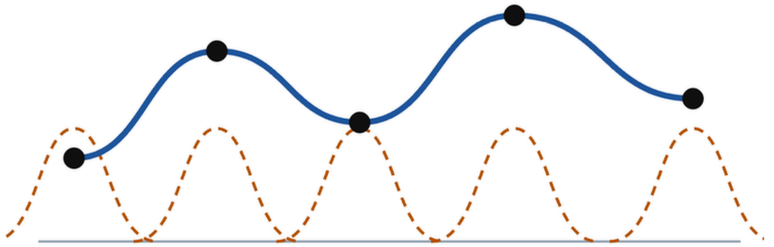
Regression/interpolation: Radial Basis Function

Approximate the latent coefficients as a weighted sum of radial kernels (Gaussian, multiquadric, thin-plate spline) centred at the P training parameters, $j = 1, \ldots, n_{train}$

$$\tilde{u}_N(\mu) \approx \sum_j \omega_j \varphi(\|\mu - \mu_j\|), \quad \omega_j \in \mathbb{R}^{n_{train}}$$

The weights follow from the interpolation conditions $\tilde{u}(\mu_i) = \tilde{u}_i$, a single linear system:

$$\Phi W = \tilde{U}, \quad \Phi_{ij} = \varphi(\|\mu_i - \mu_j\|)$$



Solid: interpolant $\tilde{u}(\mu)$. Dashed: kernels at the snapshots (dots).

Strengths

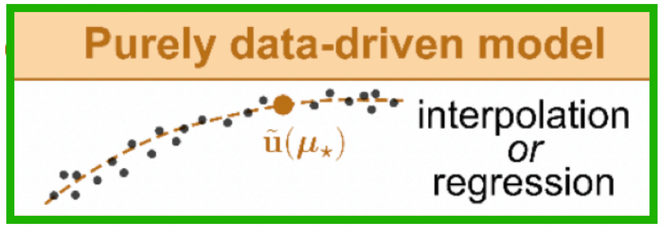
- One linear solve, exact at the snapshots
- Cheap online: P kernel evaluations
- Only the shape parameter ε to tune

Limitations

- Φ can be ill-conditioned (small ε, large P)
- No uncertainty estimate
- Degrades as the parameter dimension grows

Reduced-Order Modelling

Non-intrusive ROMs



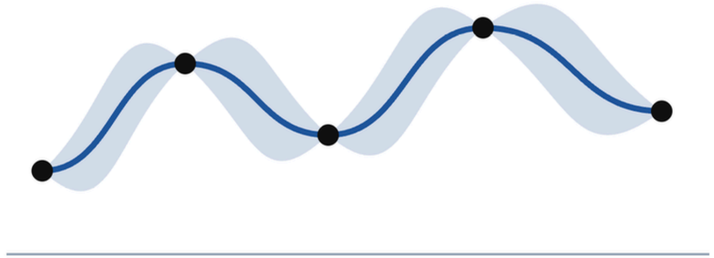
Regression/interpolation: Gaussian Process regression

Each latent coefficient is a Gaussian process over the parameter space:

$$\tilde{u}_N^k(\mu) \sim GP(0, k(\mu, \mu'))$$
$$k(\mu, \mu') = \sigma^2 e^{-\frac{\|\mu - \mu'\|^2}{2l^2}}$$

Conditioning on the P snapshots ($K = [k(\mu_i, \mu_j)]$) gives a closed-form posterior:

$$m(\mu) = k(\mu)^T (K + \sigma_n^2 I)^{-1} \tilde{u}_N^k$$
$$s^2(\mu) = k(\mu, \mu) - k(\mu)^T (K + \sigma_n^2 I)^{-1} k(\mu)$$



Posterior mean $m(\mu)$ and $\pm 2s(\mu)$ band, which pinches at the snapshots (dots).

Strengths

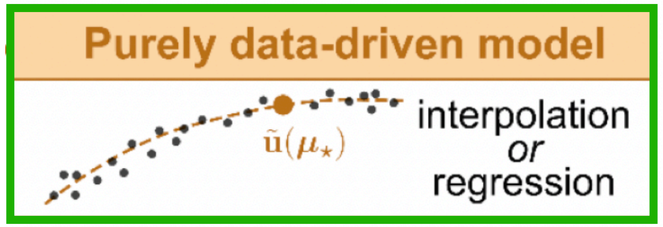
- Prediction with uncertainty $s^2(\mu)$ for free
- Hyperparameters learned from the data
- Works well with few snapshots

Limitations

- Training cost $O(P^3)$, memory $O(P^2)$
- Stationary kernels miss sharp features
- One GP per mode, or a shared kernel

Reduced-Order Modelling

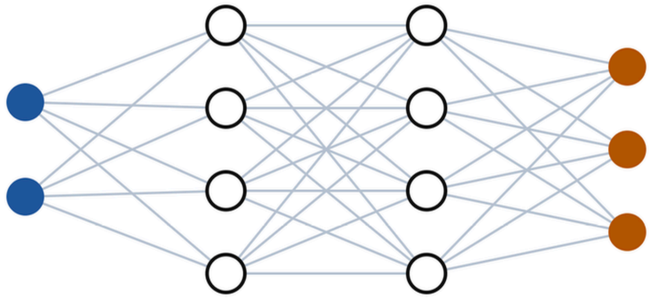
Non-intrusive ROMs



Regression/interpolation: Artificial Neural Network

A feed-forward network maps the parameters directly to the latent coefficients:

$$\tilde{u}(\mu)_N \approx NN(\mu;\theta)$$



Train with Adam (plus weight decay and a validation split) by minimising the MSE over the snapshots:

$$\theta^* = argmin(1/n_{train}) \sum_i \|\tilde{u}_N - NN(\mu_i;\theta)\|^2$$

Strengths

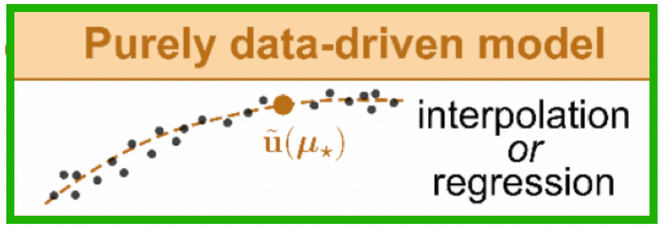
- Scales to many parameters and snapshots
- Handles strongly non-linear maps
- Fast online: a single forward pass

Limitations

- Data hungry, may overfit few snapshots
- Non-convex training, many hyperparameters
- No interpolation guarantee, no built-in UQ

Reduced-Order Modelling

Non-intrusive ROMs



$$P = n_{train}$$

	RBF	GPR	ANN
Data needed	Small	Small	Medium to large
Training	One P × P linear solve	O(P³) + likelihood optimisation	Iterative, non-convex
Online	P kernel evaluations	P kernel evaluations (mean)	One forward pass
Uncertainty	None	Yes: $s^2(\mu)$	Not built in (ensembles)
Many parameters	Degrades	Degrades (ARD helps)	Scales better
Tuning	Kernel and ε	Kernel; the rest is learned	Architecture, learning rate

Rule of thumb: few snapshots and a smooth map, use RBF or GPR. Many snapshots or many parameters, use an ANN.

Reduced-Order Modelling

Non-intrusive ROMs

Purely data-driven model



interpolation *or* regression

The full pipeline: POD + regression

Offline: once, expensive

Sample parameters
$\mu_i, \quad i = 1, ..., n_{train}$

→

Run the FOM
$u(\mu_i) \in \mathbb{R}^{\mathcal{N}}$

→

Compress with POD
$u(\mu_i)_N = \mathbf{Z}^T u(\mu_i) \in \mathbb{R}^N$

→

Train the regressor
$\mu_i \to \tilde{u}(\mu_i)(RBF, GPR, ANN)$

Online: each new parameter, cheap

New parameter
μ^*

→

Evaluate the regressor
$\tilde{u}_N(\mu^*) = regressor(\mu^*)$

→

Reconstruct
$\hat{u}(\mu^*) = \mathbf{Z}\tilde{u}_N(\mu^*) \approx u(\mu^*)$

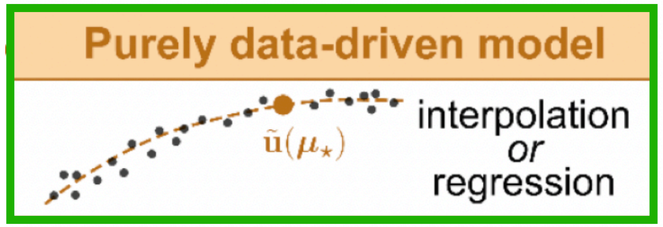
→

Output
Field or quantity of interest

Only snapshots and parameters are needed: no access to the FOM operators and no hyper-reduction. The online cost does not involve a FOM solve.

Reduced-Order Modelling

Non-intrusive ROMs

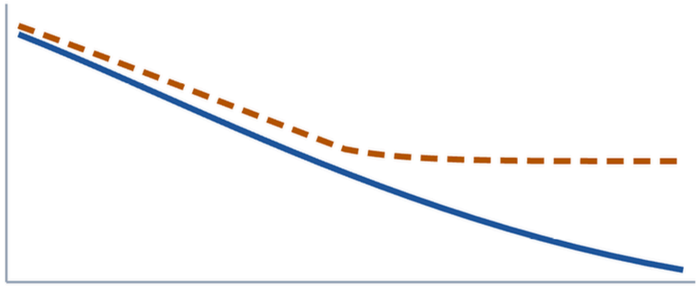


Error sources and validation

With orthonormal columns in V, the error splits into two orthogonal parts:

$$\|u - \hat{u}\|^2 = \|u - \mathbf{Z}\mathbf{Z}^T u\|^2 + \|\mathbf{Z}^T u - \tilde{u}\|$$

Projection error: set by the number of modes r and the decay of the singular values.
Regression error: set by the regressor, the number of snapshots P and their sampling.



Error vs number of modes r (log scale). Solid blue: projection error. Dashed orange: total error, which saturates at the regression error.

Sampling

Latin hypercube or Sobol points; greedy or GP-variance-driven refinement.

Validation

Split in μ into train, validation and test. Cross-validate ε, ℓ and the network size; never tune on the test set.

Metric

Relative L^2 error in the full space on unseen μ, reported next to the projection error.

Reduced-Order Modelling

Non-intrusive ROMs

Purely data-driven model



interpolation
or
regression

Time-dependent problems

In an intrusive ROM the coefficients evolve through the projected equations. Without them, there are two options:

Time as a parameter

$$\tilde{u}_N(\mu^*) = regressor(\mu^*)$$

- **Pro:** any time queried directly, no error accumulation
- **Pro:** same pipeline as before, with input (μ, t)
- **Con:** cannot extrapolate beyond the training window
- **Con:** more training pairs: P × number of time steps

Learn the dynamics

$$\tilde{u}^{n+1} = F(\tilde{u}^n, \mu), \quad n = 0, 1, ...$$

- **Pro:** can march beyond the training window
- **Pro:** closest to the projected ODE of intrusive ROMs
- **Con:** errors accumulate over the roll-out
- **Con:** stability not guaranteed, sensitive to Δt

F can be an RBF, a GP or an ANN as before; other choices are DMD, SINDy, LSTMs and neural ODEs.